Tarea Práctica — Clase 6

Calidad y testing de datos en dbt

Nicolás Blanco · Introducción a la Ingeniería de Datos · MIA 03

Proyecto base: pokemon + onecall → MotherDuck → dbt

En esta práctica se tomó como base el proyecto desarrollado en la clase 5 y se extendió con una capa completa de **testing y documentación** en dbt. Para ello se instaló `dbt-expectations`, se agregaron tests genéricos y personalizados, se documentaron modelos y columnas clave, y finalmente se validó el pipeline completo con `dbt build`. El resultado fue un proyecto funcional, documentado y con validaciones de calidad ejecutadas exitosamente.

Objetivo de la práctica

El objetivo de esta tarea fue agregar testing y documentación completa al proyecto dbt de la clase anterior. A diferencia de la clase 5, donde el foco estuvo en construir las capas `staging`, `intermediate` y `marts`, en esta práctica el foco estuvo en validar la calidad de los datos y dejar el proyecto correctamente documentado para su inspección y mantenimiento.

Para trabajar sin afectar la entrega anterior, se utilizó una copia del repositorio de clase 5 y sobre esa copia se realizaron las modificaciones requeridas para la clase 6.

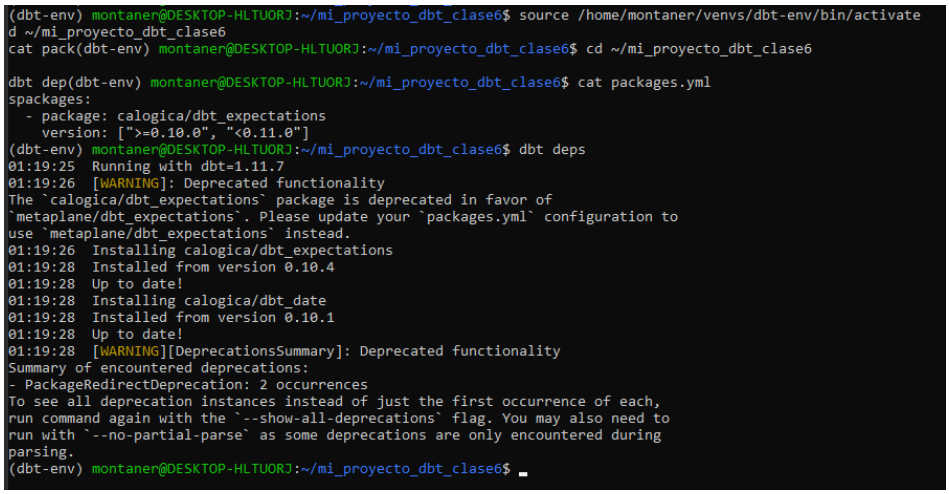
Instalación de `dbt-expectations`

El primer paso consistió en agregar el paquete `dbt-expectations` al proyecto mediante el archivo `packages.yml`. Luego se ejecutó `dbt deps` para descargar e instalar las dependencias necesarias dentro del proyecto.

```
source /home/montaner/venvs/dbt-env/bin/activate
cd ~/mi_proyecto_dbt_clase6

dbt deps
```

Esta instalación permitió incorporar tests más avanzados que los nativos de dbt, especialmente validaciones de rango numérico y controles de completitud sobre columnas específicas.



```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ source /home/montaner/venvs/dbt-env/bin/activate
d ~/mi_proyecto_dbt_clase6
cat pack(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ cd ~/mi_proyecto_dbt_clase6

dbt dep(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ cat packages.yml
spackages:
  - package: calogica/dbt_expectations
    version: [">=0.10.0", "<0.11.0"]
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ dbt deps
01:19:25  Running with dbt=1.11.7
01:19:26  [WARNING]: Deprecated functionality
The `calogica/dbt_expectations` package is deprecated in favor of
`metaplane/dbt_expectations`. Please update your `packages.yml` configuration to
use `metaplane/dbt_expectations` instead.
01:19:26  Installing calogica/dbt_expectations
01:19:28  Installed from version 0.10.4
01:19:28  Up to date!
01:19:28  Installing calogica/dbt_date
01:19:28  Installed from version 0.10.1
01:19:28  Up to date!
01:19:28  [WARNING][DeprecationsSummary]: Deprecated functionality
Summary of encountered deprecations:
- PackageRedirectDeprecation: 2 occurrences
To see all deprecation instances instead of just the first occurrence of each,
run command again with the `--show-all-deprecations` flag. You may also need to
run with `--no-partial-parse` as some deprecations are only encountered during
parsing.
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$
```

Figura 1: Instalación del paquete `dbt-expectations` en el proyecto

Configuración de tests y documentación

Una vez instalada la dependencia, se crearon archivos YAML de documentación y testing para las capas `staging`, `intermediate` y `marts`. En estos archivos se documentaron modelos y columnas clave, y además se configuraron tests genéricos como `unique`, `not_null` y `accepted_values`.

También se agregaron tests de `dbt-expectations`, principalmente con `expect_column_values_to_be_between`, para validar rangos razonables en columnas como `height`, `weight`, `base_experience`, `lat` y `lon`.

```yaml
version: 2

models:
  - name: stg_pokemon
    columns:
      - name: pokemon_id
        tests:
          - unique
          - not_null

      - name: height
        tests:
          - not_null
          - dbt_expectations.expect_column_values_to_be_between:
              arguments:
                min_value: 1
                max_value: 1000
```

De esta manera, el proyecto quedó alineado con el enfoque de la clase 6, que propone combinar tests nativos y tests extendidos dentro de archivos YAML junto con la documentación.

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ cd ~/mi_proyecto_dbt_clase6
odels.yml(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ cat models/staging/_stg__models.yml
version: 2

models:
  - name: stg_pokemon
    description: "Modelo de staging de pokemon."
    columns:
      - name: pokemon_id
        description: "Identificador único del pokemon."
        tests:
          - unique
          - not_null

      - name: pokemon_name
        description: "Nombre del pokemon."
        tests:
          - not_null

      - name: height
        description: "Altura del pokemon."
        tests:
          - not_null
          - dbt_expectations.expect_column_values_to_be_between:
              arguments:
                min_value: 1
                max_value: 1000

      - name: weight
        description: "Peso del pokemon."
        tests:
          - not_null
          - dbt_expectations.expect_column_values_to_be_between:
              arguments:
                min_value: 1
                max_value: 100000
```

Figura 2: Archivo YAML con documentación y tests configurados en la capa `staging`

Singular tests y corrección de calidad

Además de los tests genéricos, se implementaron dos **singular tests** en la carpeta `tests/`. El primero validó que no existieran Pokémon con valores negativos en `height`, `weight` o `base_experience`. El segundo verificó la consistencia entre `base_experience` y la clasificación `power_tier` generada en el modelo `obt_pokemon`.

```sql
SELECT
    pokemon_id,
    pokemon_name,
    height,
    weight,
    base_experience
FROM {{ ref('stg_pokemon') }}
WHERE height < 0
   OR weight < 0
   OR base_experience < 0
```

Durante la ejecución de los tests se detectó además un problema real de calidad en la fuente `onecall`: una fila completamente nula provocaba el fallo de las validaciones sobre `lat` y `lon`. En lugar de eliminar los tests, se corrigió el modelo `stg_weather` filtrando esa fila nula, lo que permitió mantener la validación y resolver el problema correctamente.

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ cd ~/mi_proyecto_dbt_clase6
ts/asse(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ cat tests/assert_pokemon_positive_stats.sql
ts/assert_obt_pokemon_power_tier_consistenselect
    pokemon_id,
    pokemon_name,
    height,
    weight,
    base_experience
from {{ ref('stg_pokemon') }}
where height < 0
   or weight < 0
   or base_experience < 0
cy.sql(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ cat tests/assert_obt_pokemon_power_tier_consistency.sql
select
    pokemon_id,
    pokemon_name,
    base_experience,
    power_tier
from {{ ref('obt_pokemon') }}
where
    (power_tier = 'Legendary' and base_experience < 200)
    or
    (power_tier = 'Strong' and (base_experience < 100 or base_experience >= 200))
    or
    (power_tier = 'Normal' and base_experience >= 100)
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$
```

Figura 3: Singular tests personalizados implementados para validar reglas de negocio

Ejecución final del pipeline

Una vez completadas las configuraciones, se ejecutó el proyecto completo con `dbt build`. Este comando permitió correr los modelos y los tests en el orden correcto de dependencias. La ejecución final fue exitosa, con todos los modelos creados y todos los tests aprobados.

```
dbt build
```

El resultado final confirmó que el proyecto quedó correctamente validado, sin errores ni advertencias, cumpliendo con los requerimientos de la tarea.

```
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ cd ~/mi_proyecto_dbt_clase6
(dbt-env) montaner@DESKTOP-HLTUORJ:~/mi_proyecto_dbt_clase6$ dbt build
01:21:14  Running with dbt=1.11.7
01:21:14  Registered adapter: duckdb=1.10.1
01:21:15  Found 6 models, 22 data tests, 2 sources, 741 macros
01:21:15
01:21:15  Concurrency: 1 threads (target='dev')
01:21:15
01:21:20  1 of 28 START sql view model main.stg_pokemon ................................ [RUN]
01:21:22  1 of 28 OK created sql view model main.stg_pokemon ........................... [OK in 1.60s]
01:21:22  2 of 28 START sql view model main.stg_weather ................................ [RUN]
01:21:23  2 of 28 OK created sql view model main.stg_weather ........................... [OK in 1.34s]
01:21:23  3 of 28 START test assert_pokemon_positive_stats ............................ [RUN]
01:21:23  3 of 28 PASS assert_pokemon_positive_stats ................................... [PASS in 0.24s]
01:21:23  4 of 28 START test dbt_expectations_expect_column_values_to_be_between_stg_pokemon_base_experience__10000__0  [RUN]
01:21:24  4 of 28 PASS dbt_expectations_expect_column_values_to_be_between_stg_pokemon_base_experience__10000__0  [PASS in 0.25s]
01:21:24  5 of 28 START test dbt_expectations_expect_column_values_to_be_between_stg_pokemon_height__1000__1  [RUN]
01:21:24  5 of 28 PASS dbt_expectations_expect_column_values_to_be_between_stg_pokemon_height__1000__1  [PASS in 0.22s]
01:21:24  6 of 28 START test dbt_expectations_expect_column_values_to_be_between_stg_pokemon_weight__100000__1  [RUN]
01:21:24  6 of 28 PASS dbt_expectations_expect_column_values_to_be_between_stg_pokemon_weight__100000__1  [PASS in 0.22s]
01:21:24  7 of 28 START test not_null_stg_pokemon_base_experience ....................... [RUN]
01:21:24  7 of 28 PASS not_null_stg_pokemon_base_experience ............................ [PASS in 0.22s]
01:21:24  8 of 28 START test not_null_stg_pokemon_height ............................... [RUN]
01:21:24  8 of 28 PASS not_null_stg_pokemon_height ..................................... [PASS in 0.21s]
01:21:24  9 of 28 START test not_null_stg_pokemon_pokemon_id ........................... [RUN]
01:21:25  9 of 28 PASS not_null_stg_pokemon_pokemon_id ................................ [PASS in 0.21s]
01:21:25  10 of 28 START test not_null_stg_pokemon_pokemon_name ......................... [RUN]
01:21:25  10 of 28 PASS not_null_stg_pokemon_pokemon_name ............................... [PASS in 0.22s]
01:21:25  11 of 28 START test not_null_stg_pokemon_weight ............................... [RUN]
01:21:25  11 of 28 PASS not_null_stg_pokemon_weight ..................................... [PASS in 0.21s]
01:21:25  12 of 28 START test unique_stg_pokemon_pokemon_id ........................... [RUN]
01:21:25  12 of 28 PASS unique_stg_pokemon_pokemon_id ................................ [PASS in 0.22s]
01:21:25  13 of 28 START test dbt_expectations_expect_column_values_to_be_between_stg_weather_lat__90__90  [RUN]
01:21:26  13 of 28 PASS dbt_expectations_expect_column_values_to_be_between_stg_weather_lat__90__90  [PASS in 0.21s]
01:21:26  14 of 28 START test dbt_expectations_expect_column_values_to_be_between_stg_weather_lon__180__180  [RUN]
01:21:26  14 of 28 PASS dbt_expectations_expect_column_values_to_be_between_stg_weather_lon__180__180  [PASS in 0.22s]
01:21:26  15 of 28 START test not_null_stg_weather_lat ................................... [RUN]
01:21:26  15 of 28 PASS not_null_stg_weather_lat ......................................... [PASS in 0.53s]
01:21:26  16 of 28 START test not_null_stg_weather_lon ................................... [RUN]
01:21:27  16 of 28 PASS not_null_stg_weather_lon ......................................... [PASS in 0.23s]
01:21:27  17 of 28 START sql view model main.int_pokemon_with_types ....................... [RUN]
01:21:28  17 of 28 OK created sql view model main.int_pokemon_with_types ................... [OK in 1.40s]
01:21:28  18 of 28 START sql view model main.int_weather ................................ [RUN]
01:21:29  18 of 28 OK created sql view model main.int_weather ........................... [OK in 1.38s]
01:21:29  19 of 28 START test not_null_int_pokemon_with_types_pokemon_id ................ [RUN]
01:21:30  19 of 28 PASS not_null_int_pokemon_with_types_pokemon_id ...................... [PASS in 0.25s]
01:21:30  20 of 28 START test unique_int_pokemon_with_types_pokemon_id ................... [RUN]
01:21:30  20 of 28 PASS unique_int_pokemon_with_types_pokemon_id ....................... [PASS in 0.21s]
01:21:30  21 of 28 START sql view model main.obt_weather ............................... [RUN]
01:21:31  21 of 28 OK created sql view model main.obt_weather ........................... [OK in 1.36s]
01:21:31  22 of 28 START sql view model main.obt_pokemon ............................... [RUN]
01:21:33  22 of 28 OK created sql view model main.obt_pokemon ........................... [OK in 1.35s]
01:21:33  23 of 28 START test accepted_values_obt_pokemon_power_tier__Legendary__Strong__Normal  [RUN]
01:21:33  23 of 28 PASS accepted_values_obt_pokemon_power_tier__Legendary__Strong__Normal  [PASS in 0.23s]
01:21:33  24 of 28 START test assert_obt_pokemon_power_tier_consistency ................ [RUN]
01:21:33  24 of 28 PASS assert_obt_pokemon_power_tier_consistency ...................... [PASS in 0.22s]
```

Figura 4: Ejecución final del proyecto con `dbt build`

Documentación y DAG

Finalmente, se generó la documentación del proyecto con `dbt docs generate` y luego se sirvió localmente con `dbt docs serve -host 0.0.0.0 -port 8080`. Desde esta interfaz se pudo verificar la documentación de modelos, columnas y tests, así como el grafo de dependencias del proyecto.

```
dbt docs generate
dbt docs serve --host 0.0.0.0 --port 8080
```

El DAG final mostró correctamente las fuentes `raw.pokemon` y `raw.onecall`, los modelos de `staging`, `intermediate` y `marts`, y también los tests asociados. Esta visualización permitió confirmar que la estructura del proyecto se mantuvo coherente respecto a la clase 5, pero ahora con una capa adicional de calidad y documentación.

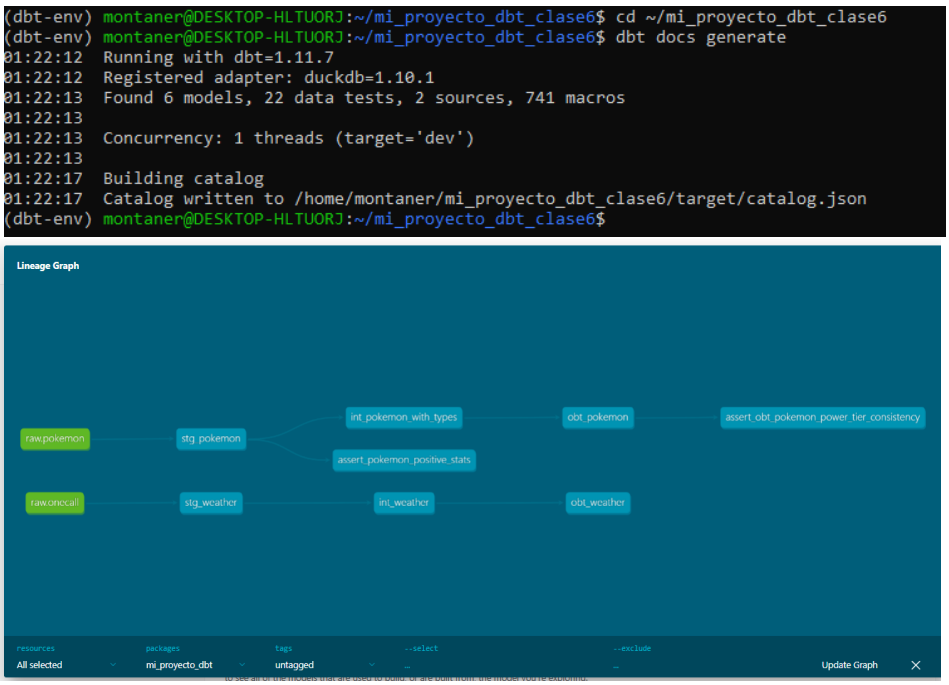


Figura 5: Lineage graph final del proyecto con modelos, fuentes y tests

Conclusión

La práctica permitió completar correctamente la evolución del proyecto iniciado en la clase 5. Mientras que en la entrega anterior el énfasis estuvo en la construcción del pipeline de transformación, en esta tarea el foco estuvo en la calidad de los datos y en la trazabilidad del proyecto.

Como resultado, se logró:

- Instalar y configurar `dbt-expectations`.
- Agregar tests genéricos y tests avanzados sobre columnas clave.
- Implementar singular tests con reglas de negocio.
- Detectar y corregir un problema real de calidad en `stg_weather`.
- Documentar modelos y columnas en archivos YAML.
- Generar el DAG y validar exitosamente todo el proyecto con `dbt build`.